



Does Federated Learning Need a Crowd?

Client Count, Not Heterogeneity, Explains FedAvg’s Failure on Real Edge Hardware

Daniel Akhabue^{1,2} David Akhabue²

¹ Qucoon, Nigeria ² Machine Learning Collective (MLC)
danakhabue@gmail.com davesakhs009@gmail.com

Abstract

Federated learning is routinely presented as a strictly beneficial alternative to centralized or local-only training on edge devices, since it preserves data sovereignty and minimises communication while supposedly matching or exceeding centralized accuracy. We test this assumption directly on real, physically heterogeneous edge hardware – an ARM64 Raspberry Pi 4 and an x86_64 gateway machine – federating a compact anomaly detection model across them with the standard FedAvg algorithm. Contrary to the common assumption, we find that naive FedAvg **consistently and substantially underperforms** both local-only training on the stronger device and simple centralized pooling, a result that holds across three independent random seeds and *worsens* the longer training continues. We then isolate the cause through a controlled experiment holding the real ARM64-vs-x86_64 data heterogeneity fixed while varying only client count. Federated accuracy improves monotonically and variance shrinks sharply as client count rises from 2 to 8, even though the underlying heterogeneity never changes. **Client count, not heterogeneity, explains the failure.** This carries direct practical weight for small edge fleets where federated learning is often adopted by default without first verifying that minimal-client federation will not actively underperform simpler alternatives.

Introduction

Edge devices intended for on-device anomaly detection – perimeter sensors, surveillance cameras, gateway nodes, drone payloads – must learn collaboratively while respecting three binding constraints:

- **Compute scarcity:** ARM-class CPUs, 4–8 GB RAM; large models are infeasible.
- **Bandwidth scarcity:** 2G/EDGE or intermittent links; streaming raw telemetry to a server is impractical.
- **Data sovereignty:** raw sensor data must not leave the device for regulatory or trust reasons.

FedAvg [1] satisfies all three by exchanging only model weight updates. Prior work demonstrates this is feasible at moderate-to-large fleet size. What is comparatively under-examined is whether this benefit holds in the *minimal-client* regime – two or three physical devices – the realistic starting point for most edge deployments before they scale, and the scenario this work investigates on real hardware.

Related Work

FL for edge/IoT anomaly detection is well-established. **FedIoT** [5] demonstrates FedAvg-based attack detection on real Raspberry Pi fleets with affordable training cost. **TinyFL** achieves F1≈97% on ESP32 industrial nodes over constrained networks [6]. A federated LSTM autoencoder on the **Marconi100 supercomputer** improves F1 from 0.39 to 0.87 across 100 compute nodes [7]. A **Pi-based smart-grid testbed** benchmarks FL feasibility via CPU/memory/power/bandwidth across seven models [8].

What is missing: none of these works explicitly isolate client count as a variable while holding genuine, hardware-derived heterogeneity fixed, nor report the specific failure mode we document here – that naive FedAvg can underperform both alternatives with only two or three real clients, regardless of how heterogeneous those clients are.

Research Question

Does FedAvg provide an accuracy benefit over local-only and centralized training when the federation has only two or three real devices? And if it underperforms, is the cause data heterogeneity between those devices, or the small number of participants itself, independent of heterogeneity?

Method

Model: MLP (11→32→16→2, ReLU), ≈700 params, <10 KB, <1 ms/sample inference.

Devices: Raspberry Pi 4 (ARM64) and an x86_64 gateway – different ISA, thermal behavior, and frequency scaling produce *naturally* non-IID telemetry without synthetic partitioning.

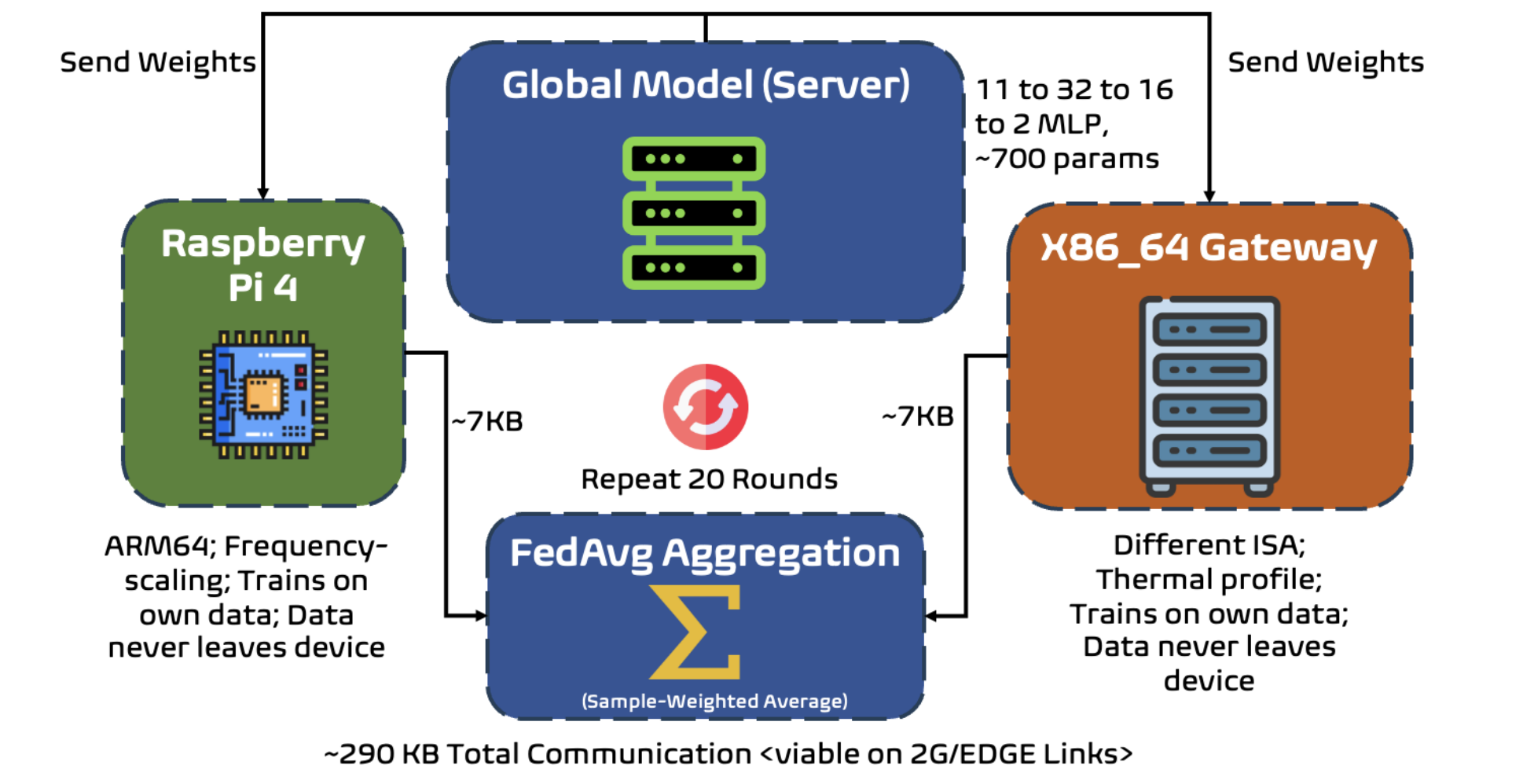


Fig. 1. Federation architecture: only weight vectors (≈7 KB/round) are transmitted; raw data stays on-device.

Matched budget: all three conditions receive an identical 100-iteration budget (20 rounds × 5 epochs), eliminating training-time as a confound.

Methods: local-only (no communication), centralized (data pooled), federated (FedAvg [1], 20 rounds).

Protocol: 3 independent random seeds; mean and std reported.

Dataset

Telemetry logged at 1 Hz from both real devices over ≈4 h each; real stress-ng anomalies injected identically on **both** (CPU spikes, memory pressure, I/O storms, disk thrash, cache pressure). 11 features: CPU%, freq, load avg (1m/5m), mem%, mem MB, swap%, disk R/W, net sent/recv.

Device	Rows	Anomaly%
Pi 4 (ARM64)	14,354	23.1%
Gateway (x86_64)	14,313	23.2%
Combined	28,667	23.2%

Near-identical anomaly rates confirm the collection schedule executed consistently on both platforms.

Experiments

Three experiments establish, test, and explain the central finding:

Exp. 1	Exp. 2	Exp. 3
Real-hardware federation	Client-count isolation	Heterogeneity sweep
2 real clients (Pi + gateway)	2–8 clients (sub-divided real data)	10 simulated clients
3 seeds	3 seeds	5 seeds
Establishes the failure	Isolates client count as cause	Reference at higher K

Results: FedAvg Fails on 2 Real Clients

Setting	Seed 42	Seed 1	Seed 7
Pi local only	0.952	0.945	0.952
Gateway local	0.851	0.848	0.836
Centralized	0.895	0.893	0.890
Federated	0.789	0.770	0.749

- Federation is **worst in every seed** by 0.10–0.14 F1 points below centralized, despite an identical matched training budget.
- F1 peaks near round 5 (≈0.83) then **degrades through round 20** (≈0.75–0.79): more rounds actively hurt performance.
- Communication cost: 290 KB total, 14.5 KB/round (viable on 2G), but bandwidth savings do not offset the accuracy cost.

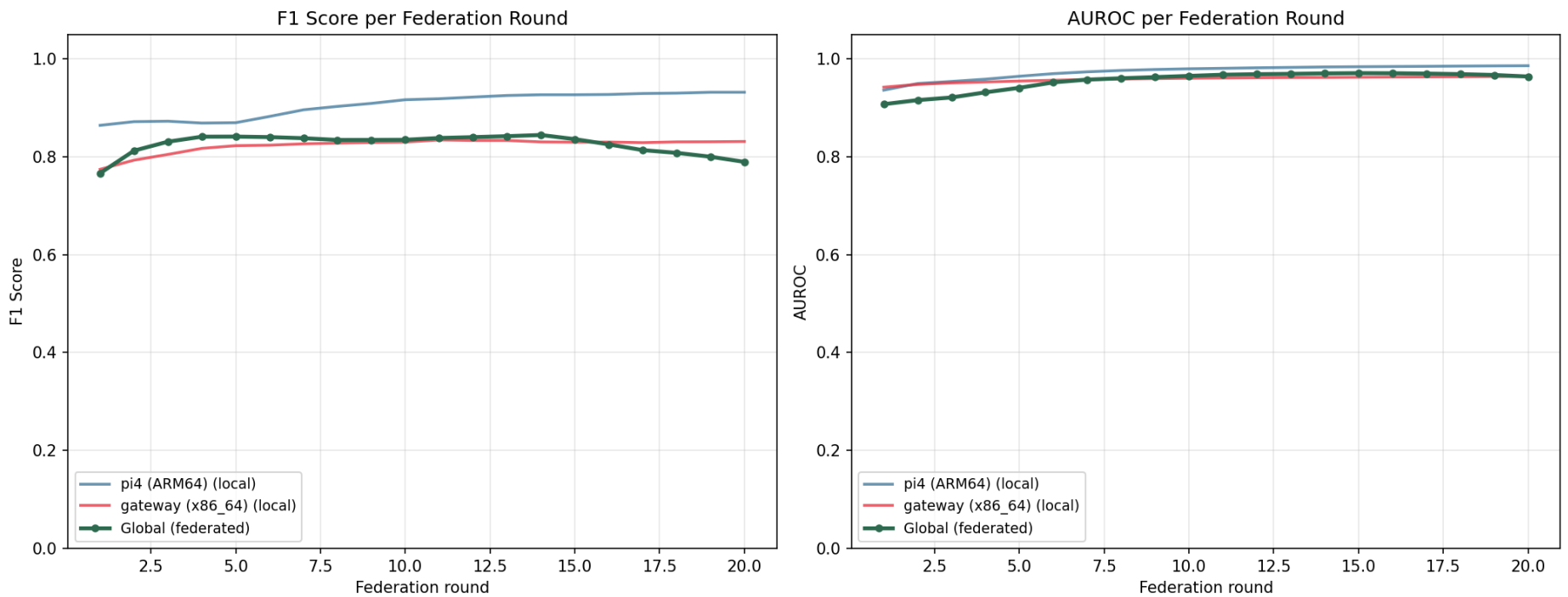


Fig. 2. Per-round F1/AUROC: federated model peaks early then degrades, while client models train independently.

Real-Hardware Validation

The result is consistent and robust: across all three seeds the ordering is always Pi-local > Centralized > Gateway-local > Federated. The systematic round-by-round degradation (peak at round 5, decline to round 20) replicates across all seeds, ruling out a single unlucky initialization as the explanation and pointing to a structural property of FedAvg’s averaging step with two clients.

Client-Count Isolation Study

To separate the contribution of data heterogeneity from client count, we split each real device’s data into k sub-clients ($k=1-4$), keeping Pi-derived and gateway-derived data strictly separated, so the genuine ARM64-vs-x86_64 heterogeneity is **held constant** while only total client count changes (2, 4, 6, 8).

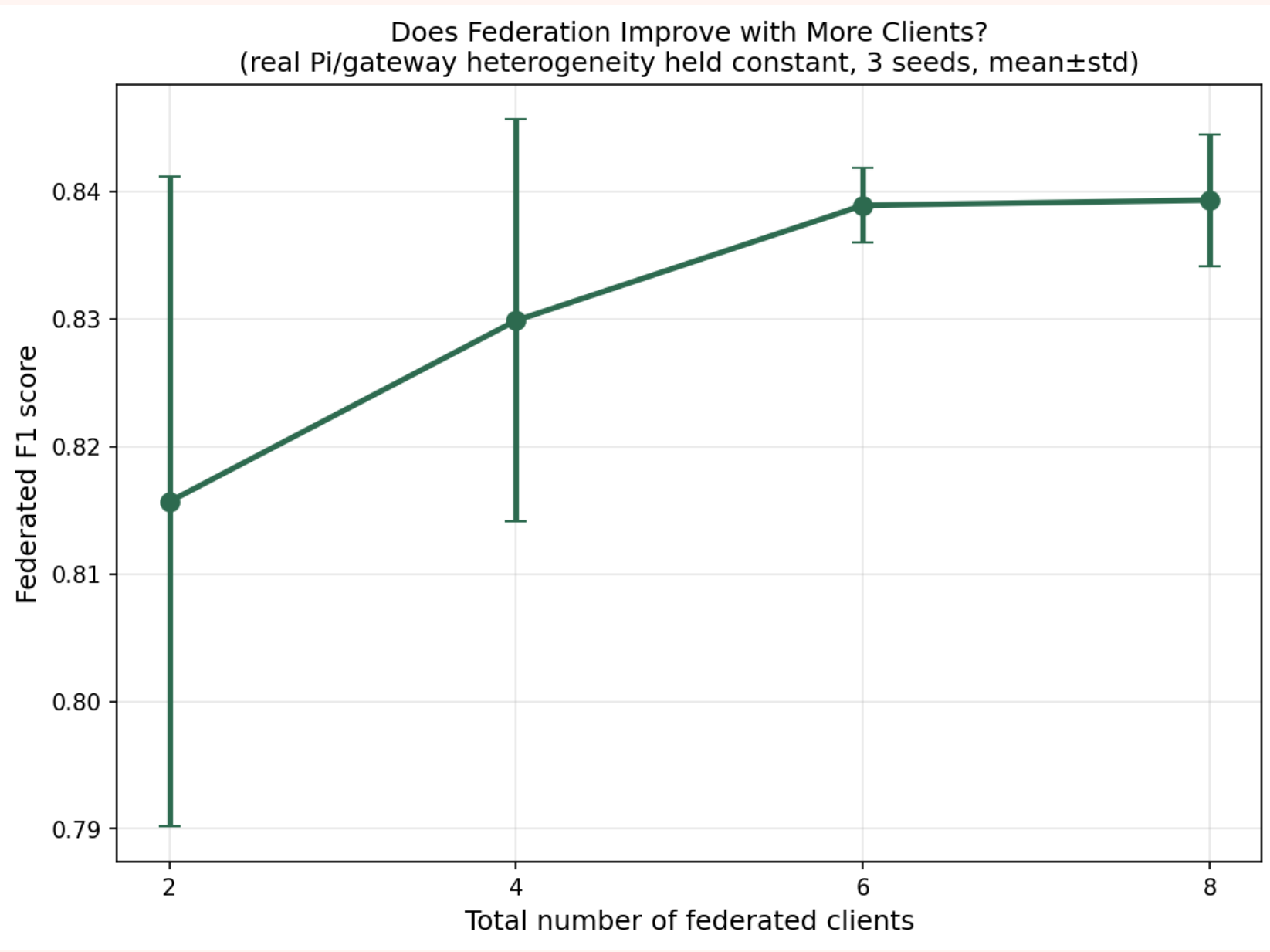


Fig. 3. Federated F1 vs. total client count, real heterogeneity fixed, 3 seeds, mean±std.

k /device	Total clients	F1 mean	F1 std
1	2	0.816	0.025
2	4	0.830	0.016
3	6	0.839	0.003
4	8	0.839	0.005

Conclusion: with heterogeneity fixed, client count alone drives the improvement – the cause is insufficient client count, not heterogeneity.

Supporting Evidence: 10-Client Sweep

As an independent reference, we partition the pooled dataset across 10 simulated clients at 5 heterogeneity levels ($\alpha \in \{100, 10, 1, 0.5, 0.1\}$, Dirichlet [2]), 5 seeds each.

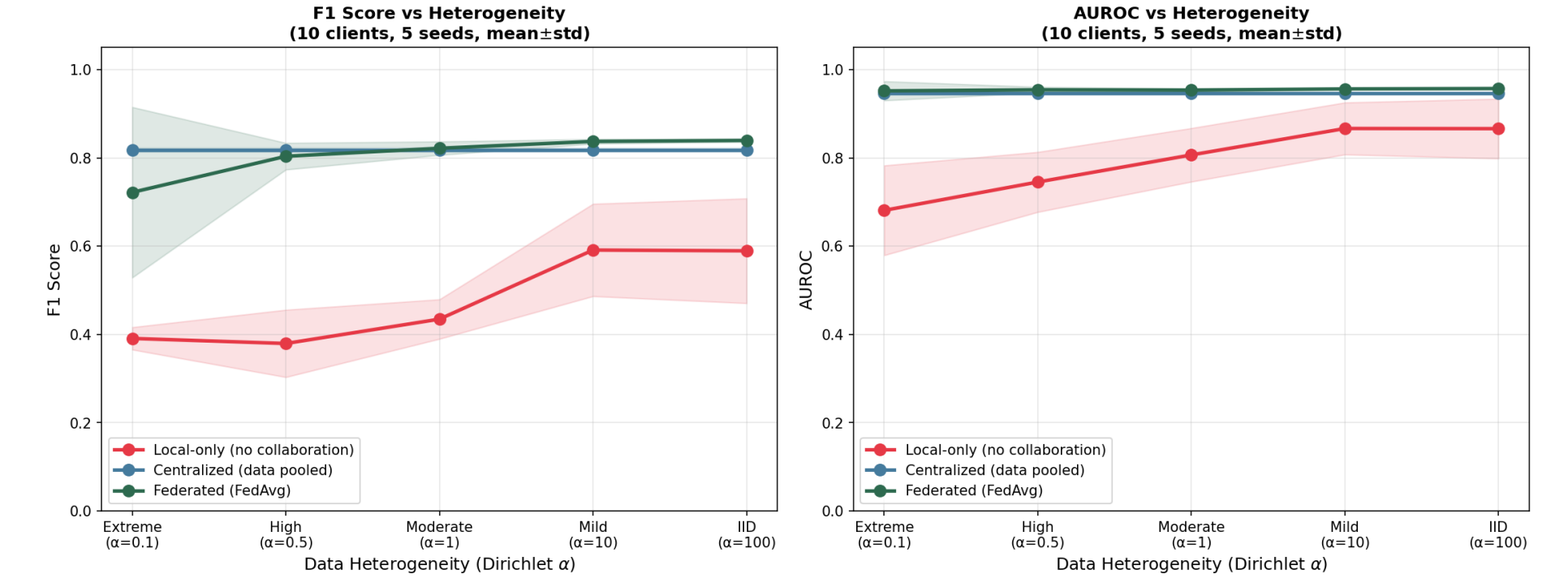


Fig. 4. At $K=10$ clients, federation *wins* across IID to moderate heterogeneity (F1 0.84 vs. 0.82 centralized), consistent with the isolation study: above the critical client-count threshold, FedAvg works as expected.

Discussion

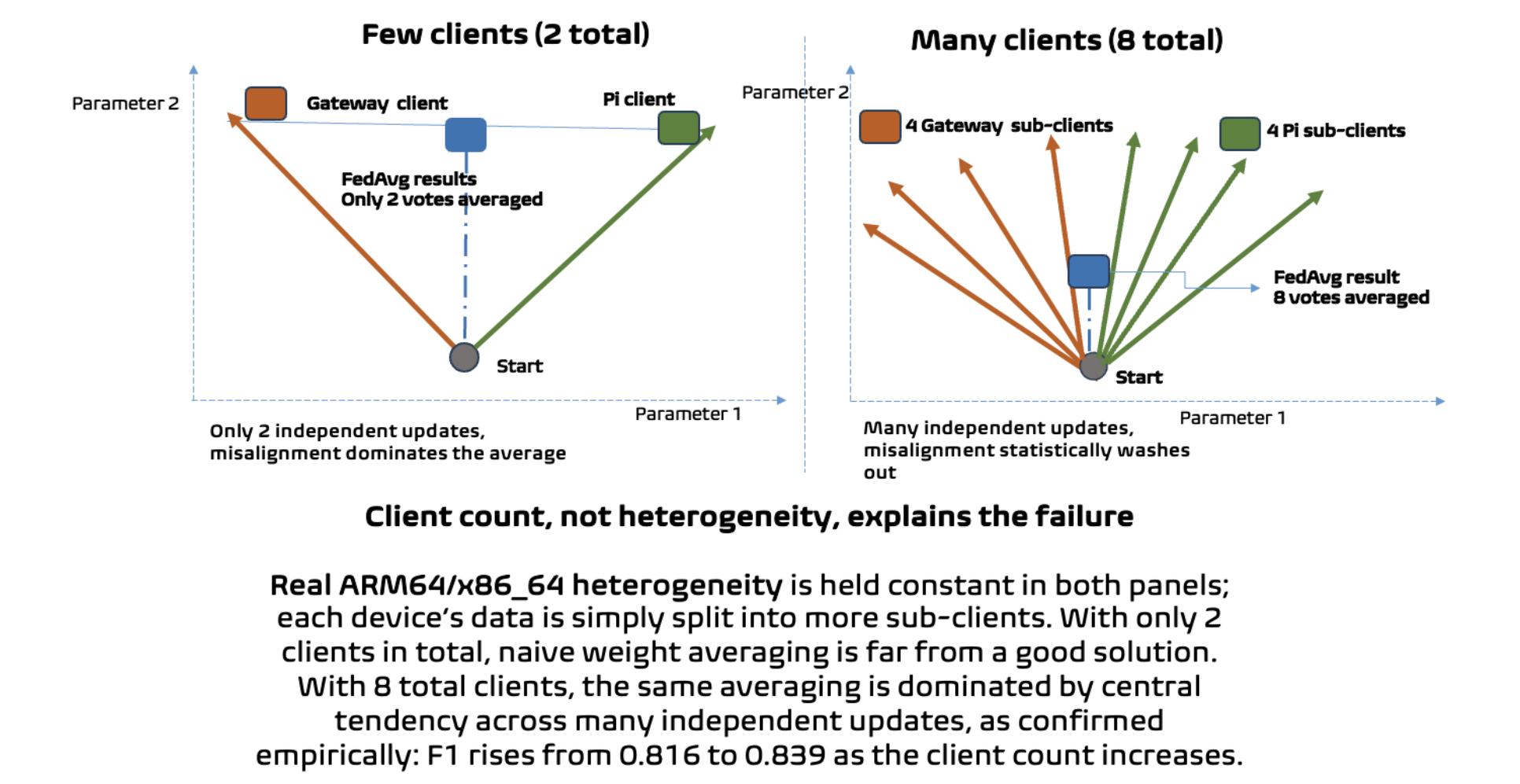


Fig. 5. Weight-space misalignment: with 2 clients, naive averaging lands far from both optima. With 8 clients, updates average toward a stable central solution.

Two independently trained neural networks solving the same task may arrive at very different weight configurations. FedAvg averages these directly – frequently violated between just two networks, producing a result resembling neither original solution. With more clients, misalignment statistically washes out, exactly the pattern the isolation experiment demonstrates.

Practical implication: for a small pilot fleet (2–5 devices), test local-only and centralized pooling as baselines before committing to FedAvg. If privacy prohibits pooling, split each device’s data into virtual sub-clients within FedAvg – our isolation study shows this recovers much of the accuracy loss without adding physical hardware.

Conclusion

Naive FedAvg substantially underperforms both alternatives on a minimal 2-device real edge deployment – confirmed across 3 seeds with matched training budgets. A controlled isolation experiment demonstrates conclusively that **client count, not heterogeneity, is the causal variable**. Practitioners should not assume FedAvg is **unconditionally beneficial on small edge fleets**: at 2–3 clients, it can actively underperform simpler alternatives.

Limitations

- Only 2 physical devices; sub-clients are data partitions, not genuinely independent hardware.
- FedProx [3] and SCAFFOLD [4] not yet tested as remedies at the 2-client scale.
- Single task domain; generalization to image/video tasks not yet evaluated.
- Energy consumption and unreliable networks not yet measured.

References

- [1] McMahan et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *AISTATS*.
- [2] Hsu et al. (2019). Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv:1909.06335*.
- [3] Li et al. (2020). Federated Optimization in Heterogeneous Networks. *MLSys*.
- [4] Karimireddy et al. (2020). SCAFFOLD: Stochastic Controlled Averaging for FL. *ICML*.
- [5] He et al. (2021). Federated Learning for IoT: On-Device Anomaly Detection. *arXiv:2106.07976*.
- [6] Marfo et al. (2025). A Framework for Tiny FL in Resource-Constrained IIoT Environments. *IEEE*.
- [7] Farooq et al. (2024). Harnessing FL for Anomaly Detection in Supercomputer Nodes. *FGCS*, 161, 673–685.
- [8] Labate et al. (2026). Towards Secure and Scalable Energy Theft Detection. *arXiv:2602.16181*.